

PostgreSQL Extensions

These are **optional extensions**. You only need them if your project requires UUIDs, cryptography, or geospatial support.

What are Extensions?

- An **extension** in PostgreSQL is like a **plugin** that adds extra features to your database.
- They are **not installed by default**, but PostgreSQL provides many official extensions.
- Example uses:
 - Generating unique IDs
 - Encrypting/Decrypting data
 - Working with maps and geographic data

You can check available extensions with:

```
SELECT * FROM pg_available_extensions;
```

To see installed extensions in your database:

```
\dx -- (in psql shell)
```

```
postgres=# SELECT * FROM pg_available_extensions;
```

name	default_version	installed_version	comment
amcheck	1.4		functions for verifying relation integrity
autoinc	1.0		functions for autoincrementing fields
bloom	1.0		bloom access method - signature file based index
bool_plperl	1.0		transform between bool and plperl
bool_plperl_u	1.0		transform between bool and plperl_u
btree_gin	1.3		support for indexing common datatypes in GIN
btree_gist	1.7		support for indexing common datatypes in GIST
citext	1.6		data type for case-insensitive character strings
cube	1.5		data type for multidimensional cubes
dblink	1.2		connect to other PostgreSQL databases from within a database dict_int 1.0
text search dictionary template for integers			
dict_xsyn	1.0		text search dictionary template for extended synonym processing
earthdistance	1.2		calculate great-circle distances on the surface of the Earth file_fdw 1.0
foreign-data wrapper for flat file access			
fuzzystrmatch	1.2		determine similarities and distance between strings
hstore	1.8		data type for storing sets of (key, value) pairs
hstore_plperl	1.0		transform between hstore and plperl
hstore_plperl_u	1.0		transform between hstore and plperl_u
hstore_plpython3u	1.0		transform between hstore and plpython3u
insert_username	1.0		functions for tracking who changed a table
intagg	1.1		integer aggregator and enumerator (obsolete)
intarray	1.5		functions, operators, and index support for 1-D arrays of integers
isn	1.2		data types for international product numbering standards
jsonb_plperl	1.0		transform between jsonb and plperl
jsonb_plperl_u	1.0		transform between jsonb and plperl_u
jsonb_plpython3u	1.0		transform between jsonb and plpython3u
lo	1.1		Large Object maintenance

1. uuid-oss Extension

Purpose

- Provides functions to generate **UUIDs (Universally Unique Identifiers)**.

UUIDs are 128-bit numbers that look like:

550e8400-e29b-41d4-a716-446655440000

- Useful as **primary keys** in distributed systems (avoids collisions better than SERIAL).

Installation

```
CREATE EXTENSION IF NOT EXISTS "uuid-oss";
```

```
postgres=# CREATE EXTENSION IF NOT EXISTS "uuid-oss";
postgres=# CREATE EXTENSION
postgres=#
```

Common Functions

- `uuid_generate_v1()` → Time-based UUID
- `uuid_generate_v4()` → Random UUID (most common)

Example

```
-- Generate a random UUID
SELECT uuid_generate_v4();
```

```
postgres=# SELECT uuid_generate_v4();
          uuid_generate_v4
-----
5167a603-e827-429e-bc39-f2886c3dd459
(1 row)

postgres=# |
```

-- Use UUID as primary key in a table

```
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name TEXT
);
```

```
postgres=# -- Use UUID as primary key in a table
postgres=# CREATE TABLE users (
postgres(#      id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
postgres(#      name TEXT
postgres(# );
CREATE TABLE
postgres=# |
```

INSERT INTO users (name) VALUES ('Alice');

```
postgres=# INSERT INTO users (name) VALUES ('Alice');
INSERT 0 1
postgres=# |
```

2. pgcrypto Extension

Purpose

- Provides **cryptographic functions**:
 - Hashing (SHA, MD5, etc.)
 - Encryption & decryption
 - Digital signatures

Installation

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

Common Functions

- `digest(data, type)` → Hashes data (e.g., SHA-256)
- `pgp_sym_encrypt(data, key)` → Symmetric encryption
- `pgp_sym_decrypt(data, key)` → Symmetric decryption

Example

-- Hash a password

```
SELECT digest('mypassword', 'sha256');
```

```
postgres=# CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE EXTENSION
postgres=# SELECT digest('mypassword', 'sha256');
              digest
-----
\x89e01536ac207279409d4de1e5253e01f4a1769e696db0d6062ca9b8f56767c8
(1 row)

postgres=# |
```

-- Encrypt and decrypt a message

```
SELECT pgp_sym_encrypt('Hello World', 'mysecretkey');
```

```
SELECT pgp_sym_decrypt(pgp_sym_encrypt('Hello World', 'mysecretkey'),
'mysecretkey');
```

```
postgres=# SELECT pgp_sym_encrypt('Hello World', 'mysecretkey');
              pgp_sym_encrypt
-----
\xcc30d040703024c5f86542b3578157dd23c0120cef84e9d034af453d7bd0f1b366b0d59927c25332e9fea70b45afa33c7d399bd4dd3a05621cf81cd5c164adee90152eebf71c23861cd7b678e58
(1 row)

postgres=# SELECT pgp_sym_decrypt(pgp_sym_encrypt('Hello World', 'mysecretkey'), 'mysecretkey');
              pgp_sym_decrypt
-----
Hello World
(1 row)

postgres=# |
```

3. PostGIS Extension

Purpose

- Adds **Geospatial (GIS) support** to PostgreSQL.
- Lets you work with **maps, coordinates, and geometry**.
- Used in apps like **Google Maps, Uber, GIS software**.

Installation

```
CREATE EXTENSION IF NOT EXISTS postgis;
```

Key Features

- Store **points, lines, polygons**
- Run **spatial queries** (distance, area, intersections)

Example

```
-- Create table with geography type
```

```
CREATE TABLE places (  
  id SERIAL PRIMARY KEY,  
  name TEXT,  
  location GEOGRAPHY(Point, 4326)  
);
```

```
-- Insert a point (Longitude, Latitude)
```

```
INSERT INTO places (name, location)  
VALUES ('Eiffel Tower', ST_SetSRID(ST_MakePoint(2.2945, 48.8584), 4326));
```

```
-- Find all places within 10 km of Paris (2.3522, 48.8566)
```

```
SELECT name  
FROM places  
WHERE ST_DWithin(location, ST_MakePoint(2.3522, 48.8566)::geography, 10000);
```

How to Enable Extensions (pgAdmin & psql)

Using pgAdmin

Open **pgAdmin** → Connect to your database.

Go to your **database** → Right-click → **Query Tool**.

Run:

```
CREATE EXTENSION "uuid-ossf";
```

```
CREATE EXTENSION pgcrypto;
```

```
CREATE EXTENSION postgis;
```

3. Now you can use the new functions.

Using psql (SQL Shell)

```
\c mydatabase -- connect to database
```

```
CREATE EXTENSION "uuid-ossf";
```

```
CREATE EXTENSION pgcrypto;
```

```
CREATE EXTENSION postgis;
```

Summary (Easy to Remember)

- **uuid-ossf** → Generate unique IDs.
- **pgcrypto** → Secure your data (hash/encrypt).
- **postgis** → Work with maps & locations.

Examples of PostgreSQL Extensions

1. uuid-oss Unique IDs in an E-commerce Website

Imagine you are building an **Amazon-like store**.

Instead of using SERIAL numbers (1, 2, 3...), you use **UUIDs** for order IDs.

Why?

- Orders may come from **different servers/countries** at the same time.
- UUIDs **never clash** → safe for global systems.

```
CREATE EXTENSION IF NOT EXISTS "uuid-oss";
```

```
CREATE TABLE orders (  
  order_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  customer_name TEXT,  
  product_name TEXT,  
  quantity INT  
);
```

```
INSERT INTO orders (customer_name, product_name, quantity)  
VALUES ('Alice', 'Laptop', 1);
```

```
SELECT * FROM orders;
```

Output Example:

1

SELECT * FROM orders;

Data Output

Messages

Notifications

≡

+

▼

▼

SQL

	order_id [PK] uuid	customer_name text	product_name text	quantity integer
1	97deaef1-aaf4-4df3-aa19-23837ac92c12	Alice	Laptop	1

2. pgcrypto Securing User Passwords

Imagine you're building a **banking app**.

You **must not store passwords in plain text**. Instead, you **hash them**.

Why?

- If hackers steal the database, passwords remain **unreadable**.

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  username TEXT,  
  password_hash BYTEA  
);
```

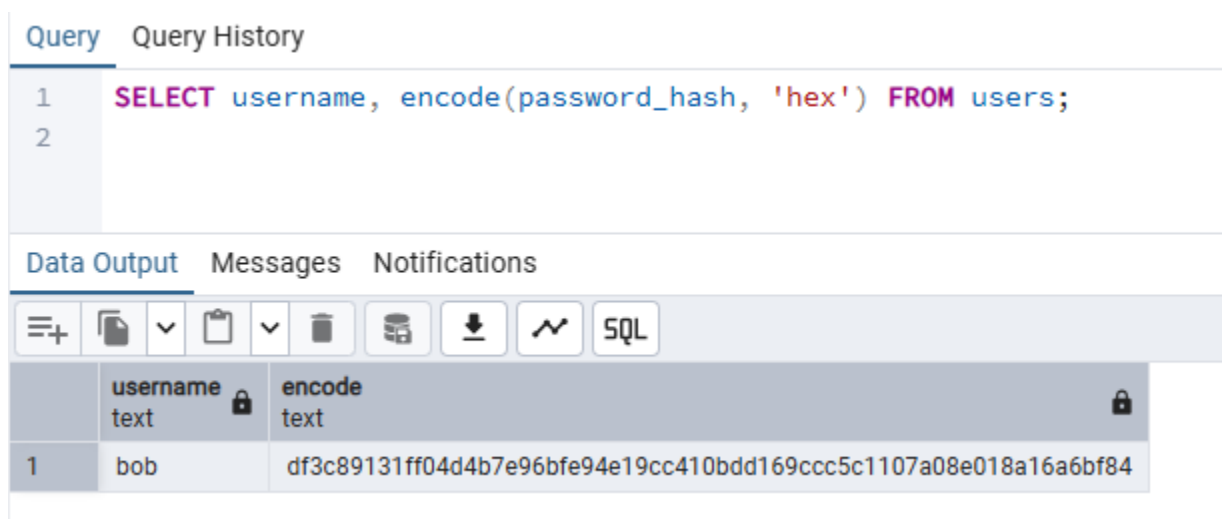
```
-- Store hashed password
```

```
INSERT INTO users (username, password_hash)  
VALUES ('bob', digest('MySecurePass123', 'sha256'));
```

```
-- Check stored value
```

```
SELECT username, encode(password_hash, 'hex') FROM users;
```

Output Example:



The screenshot shows a database query interface with a 'Query' tab selected. The query entered is: `SELECT username, encode(password_hash, 'hex') FROM users;`. Below the query editor, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, displaying a table with two columns: 'username' (text) and 'encode' (text). The table contains one row with the username 'bob' and a long hexadecimal hash string: 'df3c89131ff04d4b7e96bfe94e19cc410bdd169ccc5c1107a08e018a16a6bf84'.

	username text	encode text
1	bob	df3c89131ff04d4b7e96bfe94e19cc410bdd169ccc5c1107a08e018a16a6bf84

3. PostGIS Food Delivery App (like Swiggy / Uber Eats)

Imagine you are building a **delivery app**.

You need to find the **restaurants nearest to the customer**.

Why?

- Location-based queries are required.
- PostGIS makes this **very easy**.

```
CREATE EXTENSION IF NOT EXISTS postgis;
```

```
-- Create restaurants table with location
```

```
CREATE TABLE restaurants (
```

```
  id SERIAL PRIMARY KEY,
```

```
  name TEXT,
```

```
  location GEOGRAPHY(Point, 4326)
```

```
);
```

```
-- Insert some restaurants (Longitude, Latitude)
```

```
INSERT INTO restaurants (name, location)
```

```
VALUES
```

```
('Pizza Hut', ST_SetSRID(ST_MakePoint(77.5946, 12.9716), 4326)), -- Bangalore
```

```
('Dominos', ST_SetSRID(ST_MakePoint(77.6400, 13.0000), 4326)); -- Nearby
```

```
-- Find restaurants within 5 km of user's location (77.6000, 12.9800)
```

```
SELECT name
```

```
FROM restaurants
```

```
WHERE ST_DWithin(location, ST_MakePoint(77.6000, 12.9800)::geography, 5000);
```

Output Example:

```
name
```

```
-----
```

```
Pizza Hut
```

```
Dominos
```

Summary

- **uuid-oss** → Used in **global e-commerce systems** for safe unique IDs.
- **pgcrypto** → Used in **banking, login systems** to secure passwords & sensitive data.
- **PostGIS** → Used in **maps, delivery apps, transport apps** for location-based queries.

Download Complete SQL & PostgreSQL Notes + Practice Files

You can access the full set of **SQL/PostgreSQL notes** along with practice datasets and queries from this GitHub repository:

👉 [SQL-resources-and-tutorials by akshay-dhage](#)